

```
// AISC PRACTICAL EXAM MASTER CODES - READABLE MACRO PRINT
// 24CE1045 Aaryan Choube | Batch A/A1 | Python (All Exp) &
PROLOG (Exp 5)
```

```
// [EXP 2] water_jug.py (Water Jug State Space Search in Python)
```

```
from collections import defaultdict
jug1, jug2, aim = 4, 3, 2
visited = defaultdict(lambda: False)

def waterJugSolver(amt1, amt2):
    if (amt1 == aim and amt2 == 0) or (amt2 == aim and amt1 == 0):
        print(f"Goal Reached: ({amt1}, {amt2})")
        return True
    if not visited[(amt1, amt2)]:
        print(f"State: ({amt1}, {amt2})")
        visited[(amt1, amt2)] = True
        return (waterJugSolver(0, amt2) or waterJugSolver(amt1, 0) or
                waterJugSolver(jug1, amt2) or waterJugSolver(amt1, jug2)
                or
                waterJugSolver(amt1 + min(amt2, (jug1 - amt1)), amt2 -
min(amt2, (jug1 - amt1))) or
                waterJugSolver(amt1 - min(amt1, (jug2 - amt2)), amt2 +
min(amt1, (jug2 - amt2))))
    return False
```

```
waterJugSolver(0, 0)
```

```
// [EXP 3] tictactoe_minimax.py (Tic-Tac-Toe Minimax Algorithm in
Python)
```

```
import math

def evaluate(b):
    lines =
[[0,1,2],[3,4,5],[6,7,8],[0,3,6],[1,4,7],[2,5,8],[0,4,8],[2,4,6]]
    for x, y, z in lines:
        if b[x] == b[y] == b[z] and b[x] != ' ': return 10 if b[x] == 'X'
else -10
    return 0

def minimax(b, depth, is_max):
    score = evaluate(b)
    if score != 0 or ' ' not in b: return score
    if is_max:
        best = -math.inf
        for i in range(9):
            if b[i] == ' ':
                b[i] = 'X'; best = max(best, minimax(b, depth+1, False));
b[i] = ' '
        return best
    else:
        best = math.inf
        for i in range(9):
            if b[i] == ' ':
                b[i] = 'O'; best = min(best, minimax(b, depth+1, True));
b[i] = ' '
        return best

def find_best_move(b):
    best_val, best_move = -math.inf, -1
    for i in range(9):
        if b[i] == ' ':
            b[i] = 'X'; val = minimax(b, 0, False); b[i] = ' '
            if val > best_val: best_val, best_move = val, i
    return best_move
```

```
// [EXP 4] astar_search.py (Informed A* Search Algorithm in Python)
```

```
class Graph:
    def __init__(self, adj, h):
        self.adj, self.h = adj, h
    def a_star(self, start, stop):
        open_set, closed_set = {start}, set()
        g, parents = {start: 0}, {start: start}
        while open_set:
            n = min(open_set, key=lambda v: g[v] + self.h.get(v,
float('inf')))
            if n == stop:
                path = []
                while parents[n] != n: path.append(n); n = parents[n]
                path.append(start); path.reverse(); return path
            open_set.remove(n); closed_set.add(n)
            for m, weight in self.adj.get(n, []):
                if m in closed_set: continue
                if m not in open_set: open_set.add(m)
                tentative_g = g[n] + weight
                if tentative_g < g.get(m, float('inf')):
                    g[m], parents[m] = tentative_g, n
        return None
```

```
// [EXP 5] prolog_programs.pl (SWI-Prolog Logic Programs - Weather,
Family, Temp)
```

```
% Weather Rules
weather(phoenix, summer, hot). weather(la, summer, warm).
weather(phoenix, winter, warm).
```

```
warmer_than(C1, C2) :- weather(C1, summer, hot), weather(C2, summer,
warm).
```

```
% Family Tree
```

```
parent(joe, jane). parent(harry, carl). parent(meg, jane).
parent(jane, anne). parent(carl, ralph). parent(hazel, harry).
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
ancestor(X, Y) :- parent(X, Y).
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
```

```
% Temperature Conversion
```

```
centigrade_to_fahrenheit(C, F) :- F is (C * 9 / 5) + 32.
is_below_freezing(C) :- C < 0.
```

```
// [EXP 6] perceptron_slp_mlp.py (Single-Layer & Multilayer
Perceptron in Python)
```

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
```

```
X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
slp = Perceptron(max_iter=1000, random_state=42).fit(X_train, y_train)
print("SLP Accuracy:", accuracy_score(y_test, slp.predict(X_test)))
```

```
mlp = MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=500,
random_state=42).fit(X_train, y_train)
print("MLP Accuracy:", accuracy_score(y_test, mlp.predict(X_test)))
```

```
// [EXP 7] mcculloch_pitts.py (McCulloch-Pitts Neuron Model for
Logic Gates)
```

```
def mcculloch_pitts(inputs, weights, threshold):
    net = sum(x * w for x, w in zip(inputs, weights))
    return 1 if net >= threshold else 0
```

```
# AND Gate (weights=[1, 1], threshold=2)
# OR Gate (weights=[1, 1], threshold=1)
# NOT Gate (weight=[-1], threshold=0)
print("AND(1, 1) =", mcculloch_pitts([1, 1], [1, 1], 2))
print("OR(0, 1) =", mcculloch_pitts([0, 1], [1, 1], 1))
print("NOT(1) =", mcculloch_pitts([1], [-1], 0))
```

```
// [EXP 8] fuzzy_operations.py (Fuzzy Set Operations & Relations in
Python / NumPy)
```

```
import numpy as np
```

```
A = np.array([0.2, 0.5, 0.8, 1.0])
B = np.array([0.3, 0.7, 0.6, 0.4])
```

```
union = np.maximum(A, B)
intersection = np.minimum(A, B)
comp_A = 1 - A
diff_AB = np.minimum(A, 1 - B)
```

```
# Cartesian Product R = A x B
R = np.minimum.outer(A, B)
```

```
# Max-Min Composition T = R o S
```

```
S = np.array([[0.8, 0.4], [0.6, 0.7], [0.5, 0.9], [0.3, 0.8]])
T = np.zeros((R.shape[0], S.shape[1]))
for i in range(R.shape[0]):
    for j in range(S.shape[1]):
        T[i, j] = max(min(R[i, k], S[k, j]) for k in range(R.shape[1]))
```

```
print("Union:", union, "\nIntersection:", intersection)
print("Cartesian Product R:\n", R, "\nMax-Min Composition T:\n", T)
```

```
// [EXP 9] text_cleaning_udf.py (Text Preprocessing & Cleaning UDF
for Generative AI)
```

```
import re, string
```

```
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'http\S+|www\S+', '', text)
    text = re.sub(r'<.*?>', '', text)
    text = text.translate(str.maketrans('', '', string.punctuation))
    text = re.sub(r'\d+', '', text)
    return re.sub(r'\s+', ' ', text).strip()
```

```
sample = "<p>Welcome to AISC Lab 2026! Visit http://ai.org & learn AI &
NLP.</p>"
```

```
cleaned = clean_text(sample)
tokens = cleaned.split()
print("Cleaned:", cleaned)
print("Tokens :", tokens)
```